

AI PDF Document Assistant (RAG) — System Specification & Technical Documentation

This document provides a comprehensive technical architecture and system specification for the AI PDF Document Assistant (DocuMind AI) platform. It outlines functional requirements, security boundaries, API interactions, technology stacks targeting Python 3.13, and the machine learning / Retrieval-Augmented Generation (RAG) pipeline.

1. Functional Requirements

Authentication & Session Management

- **User Registration & Login:** Identity authentication using bcrypt salted password hashing. Plaintext credential storage is strictly prohibited.
- **Stateless JWT Authentication:** Stateless JSON Web Tokens (JWT) signed via cryptographic secret with an explicit 12-hour expiration window (`ACCESS_TOKEN_EXPIRE_HOURS=12`).
- **Strict Cookie Storage:** Tokens are stored exclusively in secure, HttpOnly cookies (`SameSite=Lax`, `Max-Age=43200`). Client-side storage (such as `localStorage` or `sessionStorage`) is prohibited to prevent Cross-Site Scripting (XSS) credential theft.
- **Multi-Tenant User Isolation:** Enforced data partitioning ensuring all uploaded documents, embeddings, and chat threads are strictly scoped to the authenticated user ID.
- **12-Hour Session Caching:** Valid, unexpired cookies prevent redundant authentication calls across page reloads and browser sessions.

Document Ingestion & Automated Review

- **PDF Ingestion:** Multi-file drag-and-drop uploader supporting PDF files with type validation and size limits (up to 25MB).
- **Lifecycle Tracking:** Document processing states are tracked in real-time across states: `Uploaded` → `Parsing` → `Chunking` → `Embedding` → `Ready`.
- **Automated Document Review:** Immediately upon ingestion completion, an automated AI background process extracts an executive 3-bullet summary, key topics, and 3 starter prompt questions.
- **Cascading Deletion:** Removing a document initiates an atomic cascading delete across

database metadata, physical disk/S3 storage, and corresponding vector indices.

Conversational RAG & Real-Time Querying

- **Explicit Submission Model:** Queries execute strictly on Enter keypress or clicking Send (with Shift + Enter reserved for multi-line inputs). Unnecessary debounce delays are omitted from the chat input.
- **Submission Guard:** Input textarea and submission triggers are disabled while an AI stream is active to prevent double-submitting.
- **Real-Time Streaming:** Server-Sent Events (SSE) stream AI responses token-by-token.
- **Source Citations:** Citations include explicit page references mapped to retrieved document chunks.

State Persistence & Rehydration

- **Persistent State:** Closing the browser tab or exiting the workspace causes zero data loss.
- **Data Rehydration:** Navigating back to the dashboard reloads document inventories, processing states, summaries, and chat history from persistent storage.

2. Frontend & Backend Integration Architecture

The frontend communicates with FastAPI via standard RESTful endpoints and Server-Sent Events (SSE). Browser requests automatically attach the HttpOnly cookie by setting credentials: "include" on all fetch requests.

HTTP Method	Endpoint Route	Description	Auth / Credentials
POST	/api/auth/signup	Registers new user with salted bcrypt password hashing.	None
POST	/api/auth/signin	Validates credentials and sets 12h HttpOnly cookie.	None
POST	/api/auth/signout	Clears the authentication	HttpOnly Cookie

HTTP Method	Endpoint Route	Description	Auth / Credentials
		session cookie.	
GET	/api/documents	Retrieves all indexed documents, summaries, and status logs.	HttpOnly Cookie
POST	/api/documents/upload	Accepts multipart PDF data, chunks, and creates vector embeddings.	HttpOnly Cookie
GET	/api/documents/{id}/review	Fetches AI executive summary and starter questions.	HttpOnly Cookie
GET	/api/documents/{id}/history	Retrieves persistent chat logs for a specific document.	HttpOnly Cookie
POST	/api/chat/stream	Streams real-time SSE AI responses with chunk citations.	HttpOnly Cookie
DELETE	/api/documents/{id}	Executes cascading deletion across metadata, files, and vectors.	HttpOnly Cookie

3. Technology Stack

Layer	Technology	Specification & Role
Frontend Framework	Next.js (App Router) + TypeScript	Type-safe server-rendered and client components.
UI & Styling	Tailwind CSS + shadcn/ui	Responsive layouts, modal dialogs, and workspace components.
State & Fetching	TanStack Query + Zustand	Server-state caching, invalidation, and local UI state management.
Document Display	PDF.js / react-pdf	Split-screen workspace PDF viewer with page navigation.
Backend API	FastAPI (Python 3.13)	High-performance asynchronous REST API and SSE streaming controller.
Auth & Config	pyjwt + bcrypt + pydantic-settings	Cryptographic JWT verification, password hashing, and type-safe environment configuration.
Relational Database	PostgreSQL (Async SQLAlchemy + Alembic)	ACID-compliant storage for users, metadata, and chat history.
Vector Database	ChromaDB / pgvector	Vector similarity search, chunk embeddings, and metadata indexing.

Layer	Technology	Specification & Role
Object / File Storage	Local Filesystem / AWS S3 / MinIO	Raw PDF storage linked by reference keys.

4. Machine Learning & RAG Pipeline Specifications

Document Parsing & Text Chunking

- **Parsing Engine:** PyMuPDF (fitz) or pdfplumber for layout-aware text, structural table extraction, and page number tagging.
- **Chunking Strategy:** Recursive character text splitting configured to 800–1,000 tokens per chunk with 150–200 token overlap to prevent contextual truncation across chunk boundaries.
- **Chunk Metadata Structure:**
 - document_id: Unique document identifier.
 - user_id: Document owner identity.
 - page: Exact source page number for citation generation.
 - chunk_index: Sequential chunk ordering within the document.

Local Embeddings

- **Embedding Model:** sentence-transformers running BAAI/bge-small-en-v1.5 or all-MiniLM-L6-v2.
- **Execution Architecture:** Runs directly within the Python 3.13 FastAPI runtime environment, minimizing external API costs and latency.
- **Vector Dimensions & Metric:** Dense 384-dimensional vector output scored via Cosine Similarity.

LLM Inference Gateway (NaraRouter)

- **Reasoning & Text Models:** deepseek-v4-pro-free for analytical, code, and structured reasoning; qwen-3.8-max-free for long-context analysis (up to 1M tokens).
- **Multimodal / Vision Fallback:** mistral-medium-3-5 or agnes-2.5-flash for complex visual pages, charts, and diagrams.
- **Hyperparameter Tuning:** Low temperature (0.1 to 0.2) to enforce deterministic, strictly factual generation.
- **Strict Prompt Guardrails:** The system prompt enforces responses based solely on

retrieved context chunks and mandates a fallback statement ("*I cannot find this information in the uploaded document.*") when context is absent.